Implementation and Analysis of BST, AVL and RBT Data Structures

Sebastiano Babich (172249) - Leonardo Mazzon (172587)
Damiano Netto (171948)

University of Udine, Academic Year 2025 - 2026

Contents

1 Abstract **3**

2 Introduction **3**

3 Theoretical Background **3**

3.1 Binary Search Tree (BST) 4

3.2 Adelson-Velsky Landis Tree (AVL) 4

3.3 Red Black Tree (RBT) 4

4 Implementation **6**

5 Test Cases Setup **8**

5.1 Choosing appropriate input sizes 8

5.2 Determining the number of test cases 9

5.3 Efficient Key Selection 10

5.3.1 Initializing array 10

5.3.2 Selecting test elements 10
5.4 Types of Tests Performed 11
5.4.1 Insertion Time Measurements 11
5.4.2 Rotations Count Measurements 11
5.4.3 Heights Reached Measurements 12
5.5 Command-line Interface 12
5.6 Parallel Execution 12
5.7 Data Visualization and Export 13

6 Results and Discussion **13**
6.1 Test Machine and OS 13
6.2 Insertions Tests 15
6.3 Rotations Count Tests 17
6.4 Heights Tests 18

7 Conclusions **20**

1 Abstract

The goal of the project was to implement and compare some of the data structures studied in the "Algorithms and Data Structures" course at the University of Udine, specifically standard and self-balancing binary search trees. The project focused on analyzing time complexity of insertion operation within these structures. We also analyzed the heights and rotations performed by each tree implemented, using configured test cases.

2 Introduction

Binary search trees (BST) are among the most widely used data structures, since they yield logarithmic asymptotic time in the average case of search, insertion and deletion. As the name suggests, each node in the tree has to have two children, each of which can either be another node or a null pointer. The height of a tree is defined as the number of edges in the longest path from the root to a leaf. Two examples of applications of this type of data structure, regarding Red-Black Trees (self-balancing variation of BSTs, covered in our analysis), are `std::set`[4] in C++ standard library and the `Completely Fair Scheduler`[2] used by the Linux kernel.

3 Theoretical Background

In all tree data structures, height is a fundamental property: keeping its value as small as possible helps ensure efficient operations thanks to more efficient search, insertion and deletion operations.

Trees can be divided into two categories based on how they manage their height: **binary search trees** and **self-balancing binary search trees**. Standard binary search trees do not actively control their height, whereas self-balancing binary trees use specific rules and rotation operations to keep their height as low as possible while preserving balance.

Trees of the latter kind are designed to maintain a height close to $\log_2(n)$, where n is the number of nodes in the tree.

In this project, we studied three types of trees, two of which implement self-balancing techniques.

3.1 Binary Search Tree (BST)

Standard binary search trees do not impose any restrictions on their height. Their purpose is to provide efficient search, insertion and deletion operations maintaining an ordered structure of the stored elements. However, since they do not include any mechanism to balance themselves, the tree can become unbalanced depending on the order in which nodes are inserted. For example, inserting nodes in an sorted order leads to the Worst-Case complexity (reported below in terms of asymptotic time complexity), degenerating the tree structure to a linked list. Therefore, inserting values in a random order generally produces better results and performance.

Operation	**Average Case**	**Worst Case**
Search	$\mathcal{O}(\log n)$	$\mathcal{O}(n)$
Insert	$\mathcal{O}(\log n)$	$\mathcal{O}(n)$
Delete	$\mathcal{O}(\log n)$	$\mathcal{O}(n)$

Table 1: Time complexity in BST operations

3.2 Adelson-Velsky Landis Tree (AVL)

AVL trees are a data structure implementing self-balancing binary search trees. The search operation exactly mirrors the search in a BST. Therefore, insertion and deletion initially use the corresponding BST method, followed by self-balancing rotations performed afterwards to restore the balance property.

Balance factor is a value used to check a node, understanding if it is balanced or not. Its value is calculated as the difference between the height of the left subtree and the height of the right subtree of the node. The factor of must be equal to -1, 0, or 1. If the value results outside the range, after any insertion or deletion, rotations are applied.

The goal is to maintain a logarithmic height, having an upper bound of $1.44 \cdot \log_2(n+2)$(Adelson-Velsky and Landis [1]). This guarantees better time complexity in search operations. However, as rotations provide a lot of overhead, the main drawback is insertion time. However this results necessary to achieve the aforementioned height bound.

3.3 Red Black Tree (RBT)

An RBT is a self-balacing BST where every node is colored as red or black, satisfying certain conditions. For each node we require to keep track of its color. If T is a

Operation	Average Case	Worst Case
Search	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$
Insert	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$
Delete	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$

Table 2: Time complexity in AVL operations

RBT, for all n node of T, we call $color(n)$ its color.

Let T a BST where each node has a color value associated ($color(n)$).

T is a RBT $\iff$ T satisfies the following conditions:

- For all n node in T, $color(n)$ must be either BLACK or RED (it must be a total function from the set of all nodes of T to {BLACK, RED})
- NIL leaves are considered black.
- For all red nodes n in T, n must not have a red child.
- Every path from a n node in T to its descendant NIL leaves must contain the same number of black nodes.

For read-only operations (like search, inorder visit, etc...) we have the same complexity as BST. Instead, insertion and removal operations are augmented using two new helper functions whose main purpose is to mantain the properties of the RBT, using rotations like AVL Trees.

These trees are a compromise between the previously discussed trees, as they yield efficient results even in insertion and deletion, due to their limited use of rotations during the balancing process. Although this does not guarantee we might achieve an absolute minimum possible height, their height has an upper bound of $2 \cdot \log_2(n+1)$ (Cormen et al. [3, p. 332]), which is slightly higher than the one of the AVL. Since this structure combines fast insertion, similar to a BST, with a search performance close to AVL trees, it is suitable in systems requiring frequent search, insert and deletion operation.

Operation	Average Case	Worst Case
Search	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$
Insert	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$
Delete	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$

Table 3: Time complexity in RBT operations

4 Implementation

The data structures (BST, AVL and RBT) were implemented as separate modules in the project, with similar interfaces for insertion, deletion, and search operations. Each tree maintains additional information useful for analysis, such as a function to compute height and a rotations counter. Balancing operations were separated into dedicated functions within each type of self-balancing tree to comply with the rules imposed by the data structure used.

These are the main functionalities implemented for each tree analyzed:

BST (Binary Search Trees)

- **Node implementation**: a *Node* class was implemented to store the key, references to the left and right children and a reference to the parent node. Each reference could be to another *Node*, or be null.
- **Node insertion and deletion**: insertion and deletion operations were implemented according to the standard algorithms for Binary Search Trees, while maintaining the BST ordering property.
- **Search operations**: efficient search, minimum, maximum, predecessor and successor operations were implemented by exploiting the ordering property of Binary Search Trees.
- **Tree traversals operations**: *inOrder*, *preOrder* and *postOrder* traversals were implemented iteratively in order to permit the visit of each created tree.
- **Tree rotations**: left and right rotation operations were implemented as fundamental tree transformations. These operations were used by balanced tree implementations to restore balance properties, while a rotation counter was maintained to be for experimental analysis.

AVL (Adelson-Velsky Landis Trees)

- **AVLNode implementation**: an *AVLNode* class, extending the *Node* class contained in BST, was created. Each node stores an additional *height* attribute, which was required to compute the balance factor.
- **Balance factor computation**: the balance factor of each node was calculated as the difference between the heights of its left and right subtrees. This value was used to detect unbalanced configurations.

- **Height management**: the height of each node was updated after insertions, deletions and rotations through the *update_height* function, avoiding the need to recomputing heights recursively.
- **Node insertion and deletion**: insertion and deletion operations were implemented by extending the BST algorithms and performing the required rebalancing after each update, checking the *balance_factor* calculated.
- **Tree rotations**: rotations were used as balancing operations to restore the AVL property after each update operation. These operations were partially inherited from the BST class. Single and double rotations were then selected according to the imbalance configuration detected through the balance factor.

RBT (Red-Black Trees)

- **RBTNode implementation**: an *RBTNode* class was created to extend *Node* class contained in BST. Each node stored an additional *color* field, which could be either Red or Black, used to maintain the balancing properties.
- **Color management**: utility functions were implemented to get, set and invert node colors. Missing children were considered black, simplifying the handling of boundary cases during update operations.
- **Node insertion and deletion**: insertion and deletion were implemented by extending the standard BST functions. After each operation a rebalancing procedure was performed, involving recoloring operations and tree rotations. The required operations restore the Red-Black properties while preserving the BST ordering.
- **Tree rotations**: left and right rotations inherited from BST were reused during the balancing procedures to modify the tree structure while preserving the ordering property.

5 Test Cases Setup

5.1 Choosing appropriate input sizes

The first issue to handle during the implementation of the test cases was the choice of the input sizes on which the experiments would be performed. We selected a wide range of tree sizes to make the simulation as meaningful as possible, highlighting the performance differences among the different type of trees used.

A valid range of values for the number of nodes had to allow the analysis in multiple orders of magnitude. Instead of using a linear distribution of input sizes, which could have caused issues in tests on large number of elements, a geometric progression was adopted to obtain a more uniform distribution on a logarithmic scale.

These were the parameters selected:

$$n_{min} = 1000 \qquad n_{max} = 1\,000\,000 \qquad samples = 100$$

The list of input sizes was generated through the *get_n_values* function, which computed the values of n using a geometric progression. The common ratio of the progression was defined as:

$$n_i = \left\lfloor n_{min} \cdot c^i \right\rfloor \qquad c = \left(\frac{n_{max}}{n_{min}} \right)^{(\frac{1}{samples-1})}$$

The geometric progression produced a total of 100 samples, gradually increasing from the minimum number of nodes m_{min} up to the maximum number n_{max} .

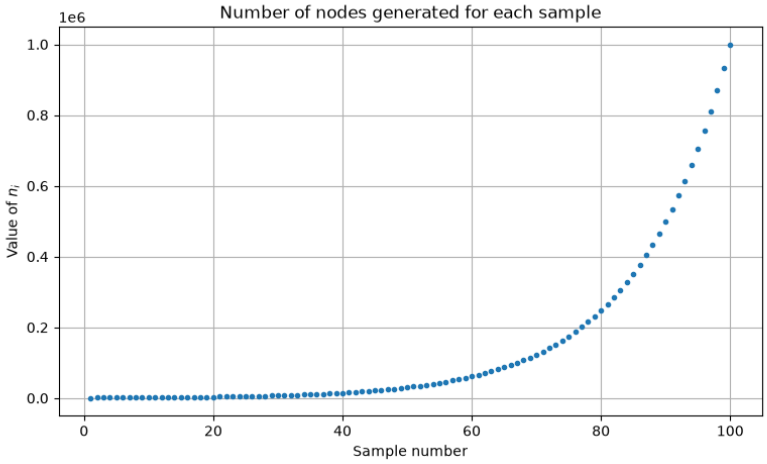

Figure 1: Distribution of the generated input sizes n_i

Then, for each n_i, the number of test cases to be performed had to be determined.

5.2 Determining the number of test cases

At first, we considered using a constant number of test cases for each n_i. However we quickly realized that this approach was not suitable: if the constant was too small, larger values of n_i would not have covered a good amount of cases. On the other hand if the constant was too big, smaller n_i would have had a lot of redundant test cases increasing execution time and resources used.

Then we adopted the following function to avoid previously mentioned issues:

$$f(n) = min + k\sqrt{n}$$

The reason for choosing $\sqrt{n}$ was that it grows slower than a linear function, avoiding an excessive increase of test cases for large values of n. It also proved to be able to cover a sufficiently amount of test cases, in order to obtain meaningful results.

The min parameter guaranteed a minimum number of test cases, even for small values of n. The k value allowed the number of generated cases to be adjusted, according to the wanted level of testing.

In the tests, we chose $min = 100$ and $k = 2$, since they provided a sufficient number of test cases both for small and large values of n.

Then, the number of test cases generated for each n_i was $100 + 2\sqrt{n_i}$.

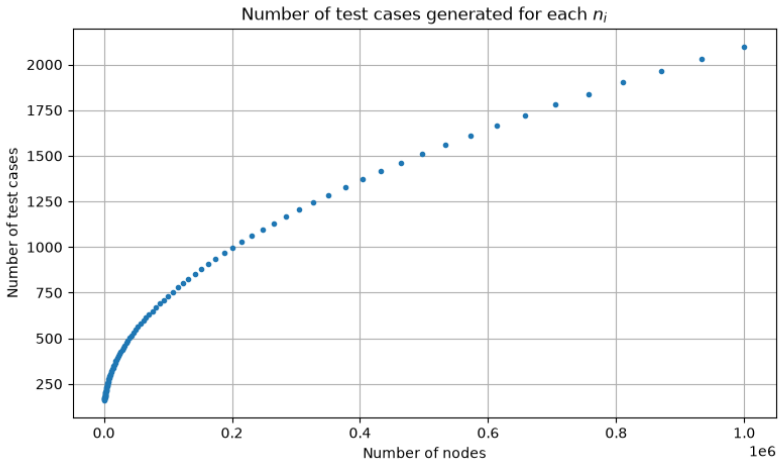

Figure 2: Number of test cases generated for each input size n_i

5.3 Efficient Key Selection

5.3.1 Initializing array

To ensure the insertion measurements accurately reflected the behavior a tree with a stable size n, we used an array containing the numbers from 0 to $n + 100 + 2\sqrt{n}$, the length of the array. After initialization, the array was shuffled using Python's *rng.shuffle* function, where *rng* is an instance of *Random* class. The first n elements were inserted into the tree to reach the target size, while the remaining served as a measurement pool. A total of $100 + 2\sqrt{n}$ insertions measurements were performed.

5.3.2 Selecting test elements

For each measurement, a new element was inserted by choosing a random index between n and the length of the array. That element was then swapped with the element at index n. To keep the tree size constant, we removed an element by picking a random index from 0 to n, subsequently swapping that selected index back into position n.

Implementing these two specific swaps was an essential step to our methodology: it preserved a uniform distribution of the test elements, making it entirely possible

for the newly inserted element to be the exact same element randomly selected for removal during that measurement cycle.

5.4 Types of Tests Performed

After analyzing the given task we identified the possibility of extending the required experiment with additional measurements related to the structural and behavioral metrics of the trees. To ensure a meaningful evaluation, we designed three distinct experimental analyses.

5.4.1 Insertion Time Measurements

We needed to analyse the execution time required to perform insertion operations on the different tree implementations. For each value of n we first generated a random tree containing n nodes. Then, we performed a sequence of random insertions random insertions followed by deletions, as specified in the "Efficient Key Selection" paragraph above. The time required for each insertion operation was measured using Python's *perf_counter* function.

The final value considered in the analysis was the median of the measured times, providing a stable estimation of the insertion costs, reducing the influence of large variations in the measurements.

5.4.2 Rotations Count Measurements

As a first additional analysis we decided to verify the number of rotations performed by self-balancing trees during update operations, since we expected it to be closely related to the execution times observed.

We introduced a *rotations_counter* variable initialized to 0 in the BST constructor, which was inherited by the other tree implementations. Since every rotation operation was performed through the BST's *rotate_left* and *rotate_right* methods, we decided to slightly modify these functions. Every time a rotation was performed, the counter was incremented by one, allowing us to keep track of the total number of rotations executed.

After each insertion and deletion, the number of rotations executed during these operation was stored. The value considered for the experiment was the number of rotations among all performed operations in 100 updates (insert+remove).

5.4.3 Heights Reached Measurements

In the second extension of the project, we analysed the height of the trees generated, comparing the behavior of each implemented data structure. The goal was to evaluate how effectively each tree maintains a low height during various updates.

To perform this analysis we used the fact that every implemented node type inherits its main features from the BST *Node* class, allowing the same height calculation method to be applied to all three data structure implementations.

We then implemented a recursive function called *height_ric*, which computes the height of a tree starting from a given *Node*. After each insertion operation we obtained the current height of the tree, starting from the root, and stored its value. At the end of each test, for each n, the maximum height obtained was considered to be the representative value.

5.5 Command-line Interface

For quick switching between different test configurations, we implemented a command-line interface based on flags.

It allows the user to choose which benchmarks to execute (insertions, rotations, and/or height measurements), the tree implementations to test (BST, AVL, and/or RBT), and the maximum number of nodes to be considered. This approach avoids modifying the source code for each experiment and makes it easy to reproduce or customize the tests through simple command-line arguments.

5.6 Parallel Execution

From our initial analysis of the number of nodes to handle, it became evident that the execution performance would gradually degrade. The total volume of operations was huge, leading to execution times that were prohibitively long and inefficient for the amount of tests we needed to run.

Therefore, it was necessary to adopt a strategy to accelerate the process. We opted to distribute the computational workload across different CPU cores using Python's `ProcessPoolExecutor`. This tool allowed us to assign the analysis of each tree structure to a different processing unit concurrently.

As a result, the overall performance improved significantly. This simple but effective optimization reduced waiting times, allowing us to execute numerous tests.

5.7 Data Visualization and Export

To clearly visualize the results of our experiments we generated graphs using Python's `matplotlib` library. In these plots, the horizontal axis represents the number of nodes in the tree, while the vertical axis represents the specific metric being evaluated, such as the median execution time, the number of performed rotations or the maximum height of the tree. Each coloured line in the graph corresponds to a different tree data structure, making it easy to visually compare their behavior as the input size increases.

Furthermore, to avoid losing any generated data or overwriting previous, the plots are saved as PNG images uniquely distinguishing each filename with a precise timestamp. Alongside the plots, the exact numerical values are also exported into CSV files. This double-saving ensured the preservation of both an immediate visual synthesis of the growth trends and the raw data necessary for any further numerical review.

6 Results and Discussion

6.1 Test Machine and OS

In our initial approach to benchmarking the insertion times of BST, AVL, and RBT, we ran our test suites on standard desktop operating systems like Windows and full-featured Linux distributions. Because our tests required precision at the microsecond scale, we quickly discovered that these environments introduced significant noise into our data. Standard operating systems manage hundreds of concurrent background processes, forcing the OS scheduler to frequently preempt our process. These relentless context switches, and suspension of our process, created unpredictable timing spikes that heavily obscured the true algorithmic performance of the trees.

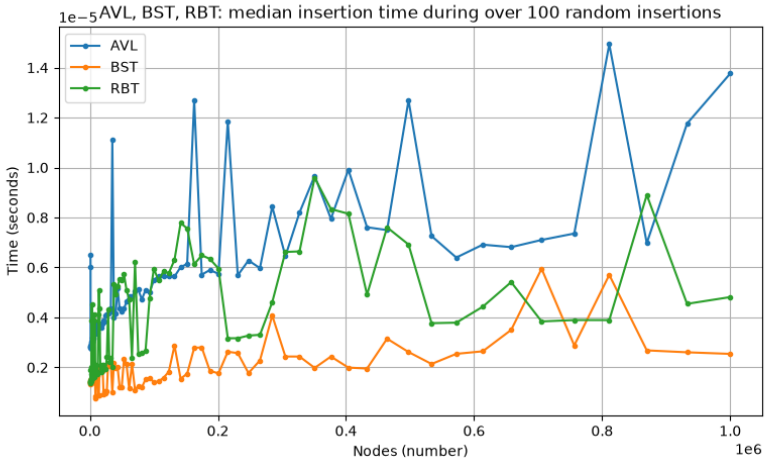

Figure 3: Example of test executed in Linux desktop

To eliminate this systemic jitter, we shifted our testing environment to a lightweight, command-line-only Linux distro (we choose Arch Linux). By stripping away the graphical user interface and unnecessary background daemons, we drastically reduced the operating system's footprint. We also executed all testing processes with maximum priority, ensuring our tests received the highest possible CPU time. This strictly controlled environment successfully suppressed external interference and scheduler interruptions, yielding highly stable and reproducible execution times.

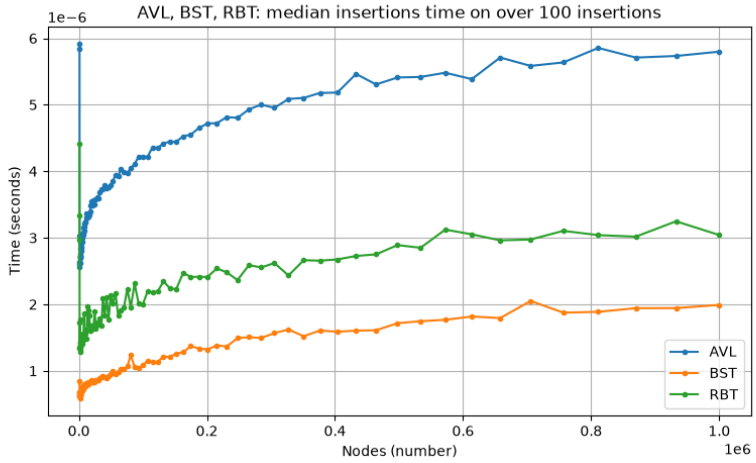

Figure 4: Example of test executed in Archlinux with maximum priority

Some details about the software used:

- OS: Archlinux Live ISO (2026.07.01)
- Kernel version: 7.0.14
- Scheduler used: Default (Completely Fair Scheduler)
- Python version: 3.14.6

Then for the hardware, we used a laptop always in charging with the following components:

- CPU: Intel Core i5-1335U
- RAM: 16GB DDR4-3200 MHz SO-DIMM Dual Channel
- Disk: Samsung SSD 980 500 GB

All the tests below are done using this hardware and software, and executing the process with maximum priority.

6.2 Insertions Tests

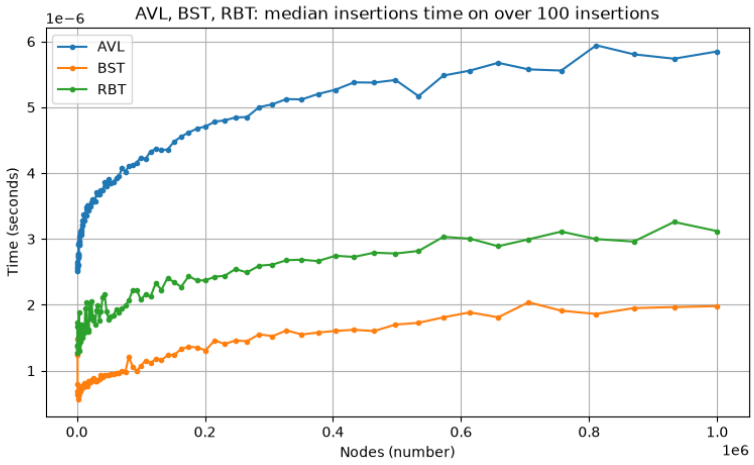

Figure 5: Insertion test of all the trees

Based on the benchmarking results in the above graph, we noticed that AVL Trees, Red-Black Trees, and standard Binary Search Trees all exhibited a distinct logarithmic growth curve. This visual confirmed that the average-case insertion time

for all three data structures would scale at $O(\log n)$. The clear vertical separation between the curves does not indicate different time complexity classes, but rather the varying **multiplicative constants** hidden within that $O(\log n)$ bound, driven primarily by their respective balancing overheads:

- The standard BST performs absolutely zero rotations and does not track balance factors. While this tree risks degrading into an $O(n)$ structure if fed sequential data, this graph clearly reflects uniformly distributed random insertions. Given a uniform data sample, a BST naturally distributes nodes well enough to remain relatively shallow on average. Stripped of all the computational weight of checking balances, recoloring, and rotating nodes, the naive BST proves highly efficient and clocks the fastest insertion times of the three.
- AVL trees enforces the strictest balancing rules to ensure the tree remaines perfectly shallow. However, this come at a steep cost during insertion, as maintaining this strictness could have require up to $O(\log n)$ structural rotations in the worst case. This heavy overhead explained why it had the highest multiplicative constant and the slowest insertion time.
- RBTs compromise by using slightly looser balancing rules. They require fewer structural changes (a maximum of two rotations per insertion), which reduces the overhead and places their performance comfortably between the AVL and BST.

To confirm the balancing overheads, we decided to keep track of all the rotations done by self-balancing threes.

6.3 Rotations Count Tests

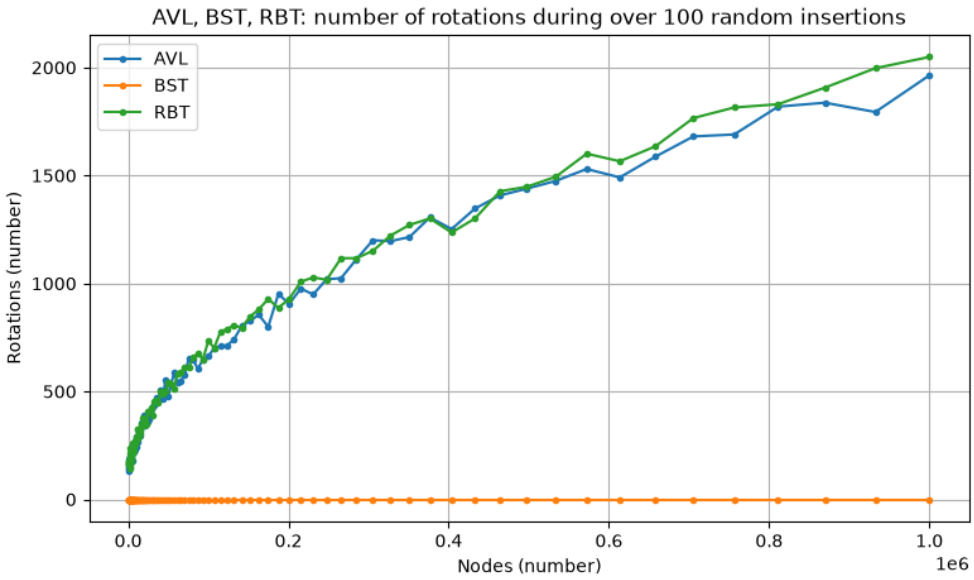

Figure 6: AVL, BST and RBT rotation test

Based on the results in the above graph, we can see that, as the number of nodes grows, the number of rotations on AVL and RBT does not differ much, however, we can see that the RBT is actually slightly better. This is due to the height handling in AVLNode, that doesn't happen in RBT due to the Color parameter.

Other than that, we observed that the number of rotations on both trees followed a logarithmic trend. To verify this observation, we used `numpy` to perform a logarithmic regression to approximate the above graph with a logarithmic function. After computing the resulting function revealed as

$$f(n) = c \cdot \log(n) + b$$

with the following constants:

$$\begin{aligned} \text{AVL:} \quad & c = 1.5465480485189718, \quad b = 85.88523518736147 \\ \text{RBT:} \quad & c = 1.5976824067351296, \quad b = 86.90539968367709 \end{aligned}$$

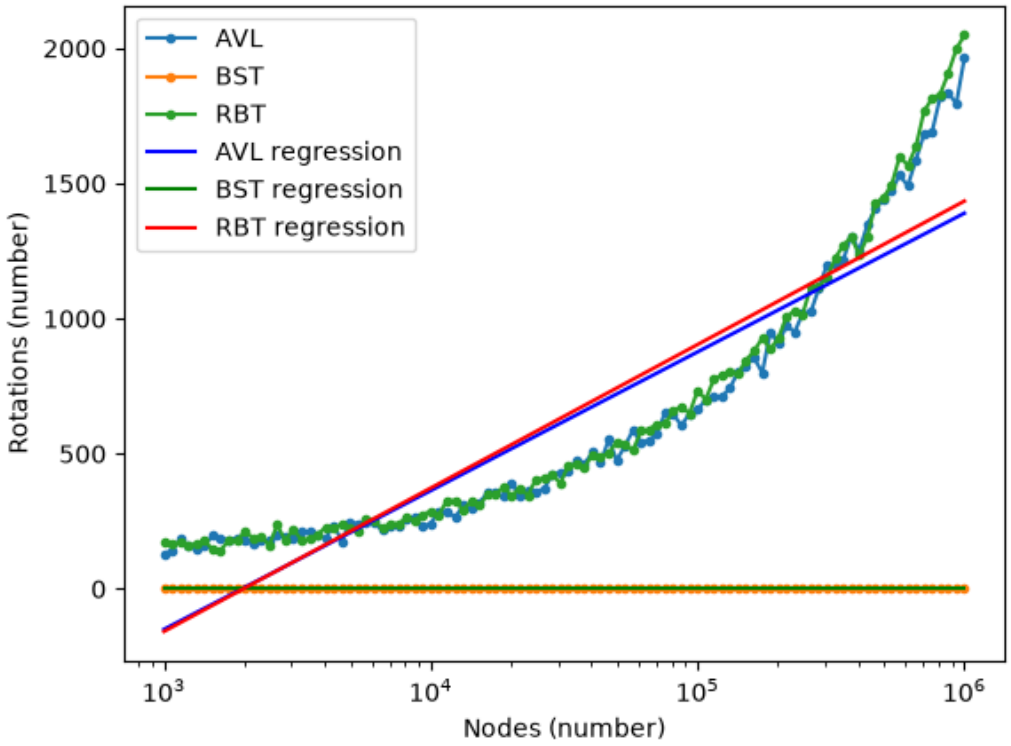

Figure 7: Rotation tests in logarithmic scale

6.4 Heights Tests

The tests we performed confirmed the theoretical expectations for each data structure implemented: as the number of nodes increased, the maximum height reached by every tree followed a logarithmic trend during random key insertions.

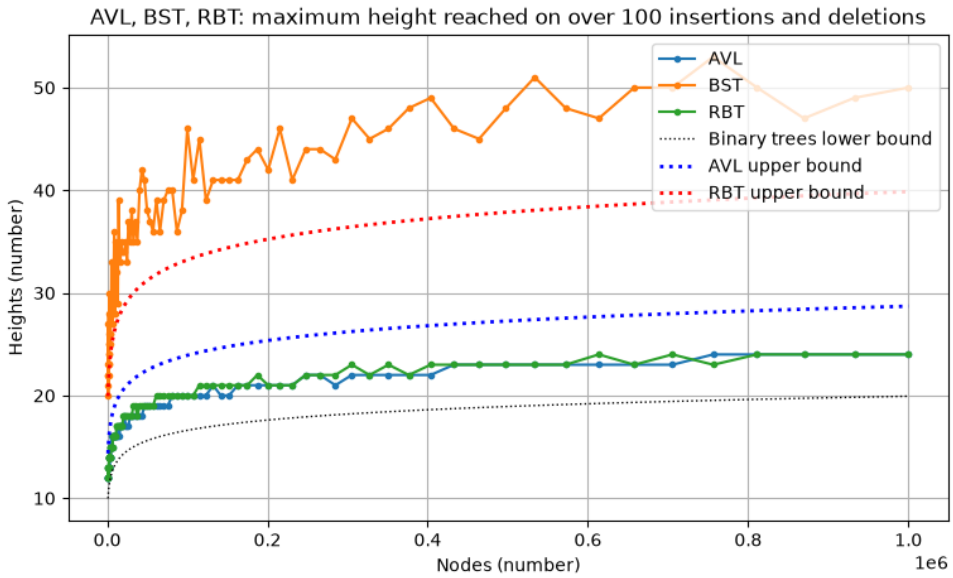

Figure 8: AVL, BST and RBT heights test

When comparing the standard Binary Search Tree with the self-balancing ones, an important difference was notable: the height of unbalanced trees grew much faster since the beginning, meanwhile AVL and RBT remained heavily compressed in terms of height reached.

At our maximum sample size of 10^6 nodes, the BST ended up being more than twice as tall as its balanced alternative versions. Meanwhile, the actual heights reached, by both self-balancing trees, remained consistently low and stable through most of the test run. This happened even though the RBT had an higher theoretical upper bound, due to a larger constant factor.

These measurements gave us a practical understanding on how effectively self-balancing tecniques are able to control height during tree updates. As a direct consequence, the computational cost for any search operation results significantly optimized in AVL and RBT structures, compared to standard BST.

7 Conclusions

The comparison demonstrated that the additional complexity required to implement self-balancing trees was justified by the significant longterm gains in efficiency and robustness offered by these data structures. While a standard BST provides a simpler implementation and execution, it remains vulnerable to worst-case scenarios making it less reliable for large-scale datasets. Instead, choosing between an AVL and a RBT allows us to optimize the system based on whether the specific application requires fast searches or a balanced mix of frequent updates and lookups.

Bibliography & Sitography

[1] Georgy Adelson-Velsky and Evgenii Landis. "An Algorithm for the Organization of Information". In: (1962).

[2] *CFS Scheduler - The Linux Kernel documentation*. Archived on `archive.org` on April 24th, 2026. URL: `https://web.archive.org/web/20260424153328/https://www.kernel.org/doc/html/latest/scheduler/sched-design-CFS.html#the-rbtree`.

[3] Thomas H. Cormen et al. *Introduction to Algorithms*. 4th ed. MIT Press, 2022.

[4] *std::set - cppreference.com*. Archived on `archive.org` on July 8th, 2026. URL: `https://web.archive.org/web/20260708070457/https://en.cppreference.com/cpp/container/set`.